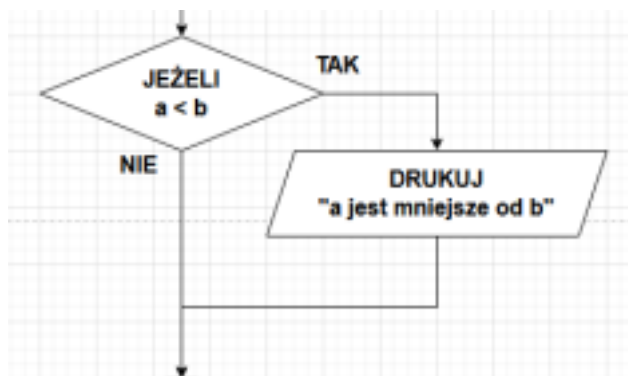


Instrukcje warunkowe

1. Instrukcja warunkowa - podstawy

Czasami chcemy rozdzielić algorytm, aby realizował różne funkcjonalności w zależności od potrzeby. Wydzielenia fragmentu kodu z głównej linii algorytmu dokonujemy z użyciem instrukcji warunkowej.



W języku C++, instrukcją warunkową jest instrukcja **if()**. Za instrukcją (wewnątrz nawiasu) powinien znajdować się zapis, który jako rozwiązanie wygeneruje wyłącznie: **true** (prawda) lub **false** (fałsz).

Bezpośrednio po instrukcji warunkowej **if()** zapisujemy blok kodu, który ma się wykonać jeśli warunek okaże się prawdą. Po wykonaniu tychże instrukcji algorytm wraca do nadrzędnej linii kodu.

Za instrukcją warunkową `if()` nie zapisujemy średnika.

Przykład filozofii zapisu poprawnego warunku poniżej. Sprawdzamy, czy zmienna „a” jest mniejsza od zmiennej „b”. Niezależnie od wartości tych zmiennych, możemy jednoznacznie odpowiedzieć na to pytanie **true** („a” jest mniejsze od „b”) lub **false** („a” nie jest mniejsze od „b”):

```
if( a < b )
{
    cout << "a jest mniejsze od b" << endl;
}
```

Dobra praktyka programistyczna.

Ponieważ elementy mniejsze są przedstawiane na osi liczbowej po lewej stronie, a elementy większe po stronie prawej, to przy używaniu operatorów mniejsze/większe zaleca się zapisywanie warunku właśnie w taki sposób.

Poniższe warunki są identyczne pod względem wykonania, oba są także poprawne pod względem programistycznym, jednakże zapis lewego może optycznie mylić co do właściwego zrozumienia intencji większa/mniejsza, tym samym łatwiej o pomyłkę w analizie kodu.

<pre>if (b > a) { cout << "a jest mniejsze od b"; }</pre>	<pre>if (a < b) { cout << "a jest mniejsze od b"; }</pre>
--	--

2. Operatory warunkowe

== - operator porównania. Jest prawdziwy, gdy argumenty po obu stronach operatora mają taką samą wartość.

Uwaga! Nie należy używać w warunkach **operatora przypisania** „=” w sytuacji, kiedy chcemy porównać ze sobą dwie wartości.

Operator „=” podstawia wartość z prawej strony do zmiennej po jego lewej stronie. Operatory „==” oraz „=” to dwa różne operatory!

!= - operator nierówności. Jest prawdziwy, gdy argumenty po obu stronach operatora mają różną wartość.

! - operator negacji logicznej. Dodany do zmiennej, argumentu lub warunku zamienia znaczenie jego stanu z **true** na **false** (i odwrotnie).

< > - operatory „mniejszy” oraz „większy”.

<= >= - operatory „mniejszy lub równy” oraz „większy lub równy”. Znak „=” zawsze musi być po prawej stronie znaku „<” lub „>”!

Uwaga na operator przypisania użyty w warunku (zamiast operatora porównania)! Autor poniższego kodu chciał sprawdzić, czy podana wartość jest równa 10. Omyłkowo użył operatora przypisania. Spowodowało to, że zmiennej „a” została przypisana wartość 10. Ponieważ jest to wartość różna od zera, zatem warunek będzie uznany za prawdziwy. Zawsze wykona się kod dla opcji **prawda**, niezależnie od wartości zmiennej „a” jaką podamy.

```
int a;
cin >> a;

if (a = 10)
{
    cout << "liczba jest równa 10";
}
```

3. Formatowanie kodu

Bezpośrednio po instrukcji warunkowej realizowany jest kod, który ma się wykonać, gdy warunek okaże się prawdziwy. Aby wskazać dokładne instrukcje, jakie mają się wykonać w takim wypadku używamy właściwego formatowania kodu. W C++ do grupowania kodu w „bloki” stosuje się parę nawiasów klamrowych { }, gdzie „{” - zaczyna blok kodu, a „}” kończy go.

Kod wewnątrz takiego bloku powinien być wcięty (wytabowany) względem instrukcji if(), aby z daleka było widać, że jest on „podległy”

względem takiej instrukcji warunkowej.

Są różne konwencje formatowania kodu z użyciem nawiasów klamrowych. Najpopularniejsze dwie: jedna zakłada, że klamra rozpoczynająca powinna być tuż za instrukcją warunkową, a zamykająca na takim samym wcięciu co instrukcja warunkowa. Druga zakłada, że klamra rozpoczynająca i zamykająca powinny być na tym samym wcięciu co instrukcja warunkowa. (moim zdaniem ten drugi zapis jest o wiele bardziej czytelny).

```
if( a == 0 ) {  
    cout << "ZERO!";  
}
```

```
if( a == 0 )  
{  
    cout << "ZERO!";  
}
```

Dla tylko jednej operacji, realizowanej pod warunkiem, można pominąć stosowanie klamr. Można nawet taką instrukcję zapisać bezpośrednio za instrukcją **if()**, w tej samej linii. Odradza się jednak taki zapis ze względu na pogorszenie czytelności kodu.

```
if( a == 0 )  
    cout << "ZERO!";
```

```
if( a == 0 ) cout << "ZERO!";
```

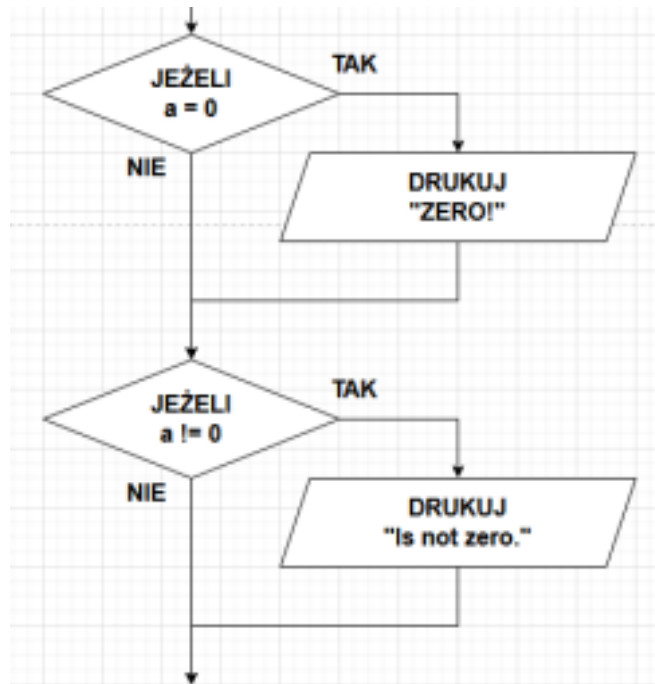
Dobra praktyka programistyczna zaleca używanie zapisu klamrowego nawet w sytuacji, gdy mamy tylko jedną instrukcję do realizacji.

4. Instrukcja else

Instrukcja warunkowa **if()**, jeśli będzie spełniona, wydzieli z głównej linii algorytmu osobny kod do wykonania. Po jego wykonaniu wraca do głównej linii algorytmu, gdzie dalej wykonuje kod podstawowy.

Jeśli chcemy obsłużyć także sytuację przeciwną, należy zapisać warunek o znaczeniu przeciwnym, tuż po zakończeniu wykonania warunku podstawowego.

Uwaga na zapis warunków przeciwnych dla operatorów „<”, „>”, „<=”, „>=”. Przykładowo: warunkiem przeciwnym do „a < b” jest „b <= a”. Nie należy zapominać, o znaku „=” w operatorze przeciwnym (lub go nie używać, gdy operator podstawowy go posiada). Analogicznie dla pozostałych operatorów.



```

if( a == 0 )
{
    cout << "ZERO!";
}

```

```

if( a != 0 )
{
    cout << "Rozne od zera";
}

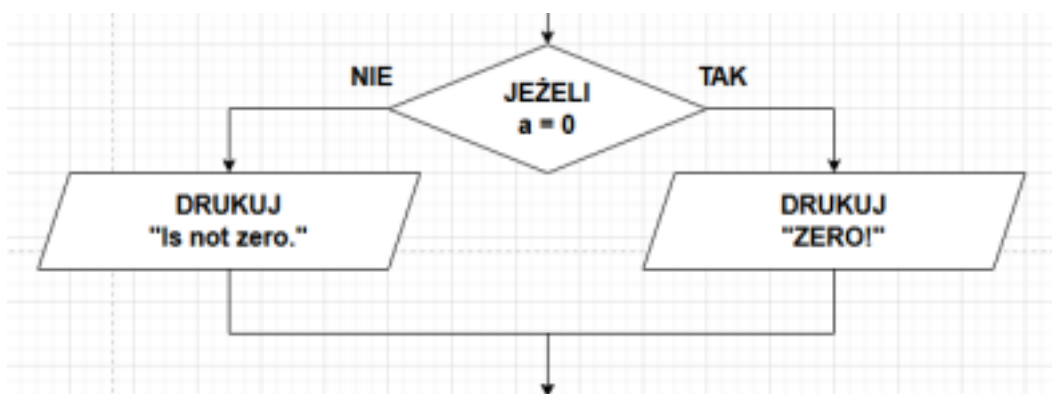
```

Powyższą konstrukcję można uprościć.

Dzięki dodaniu instrukcji **else** możemy od razu zapisać dodatkowy kod, który miałby się wykonać w przypadku warunku przeciwnego, bez konieczności zapisu takiego warunku.

Czasami **else** się pomija, bo po prostu program nie musi niczego robić, w przypadku gdy warunek nie będzie prawdziwy.

Obowiązują te same zasady formatowania kodu i stosowania klamr, co dla instrukcji **if()**.



```

if( a == 0 )

```

```

{
    cout << "ZERO!";
}
else
{
    cout << "Roznie od zera";
}

```

5. Więcej niż tylko jedna opcja do wyboru

W przypadku, gdy mamy do wykonania aż kilka opcji, to możemy rozdzielać nasz algorytm stopniowo na poszczególne fragmenty kodu. „Wyciągamy” pojedynczą opcję warunkiem, a pozostałe opcje rozważamy w **else** do tego warunku. Dla każdego „poziomu” opcji postępujemy podobnie.

```

char Kompas; // n s e w
cin >> Kompas;

if( Kompas == 'n' )
{
    cout << "Polecie";
}
else
{
    if( Kompas == 's' )
    {
        cout << "Poleknie";
    }
    else
    {
        if( Kompas == 'e' )
        {
            cout << "Wschod";
        }
        else
        {
            if( Kompas == 'w' )
            {
                cout << "Zachod";
            }
            else
            {
                cout << "Wzierny kierunek";
            }
        }
    }
}

```

Powyższy kod możemy uprościć (wizualnie) stosując zapis typu: if - else if – else. (oba zapisy mają takie samo wykonanie, chodzi wyłącznie o wygodę w zapisie kodu przez programistę).

```

char Kompas; // n s e w
cin >> Kompas;

if( Kompas == 'n' )
{
    cout << "Polecie";
}
else if( Kompas == 's' )

```

```

{
    cout << "Północ";
}
else if (kompas == 'e')
{
    cout << "Wschód";
}
else if (kompas == 'w')
{
    cout << "Zachód";
}
else
{
    cout << "Nieznany kierunek";
}
}

```

6. Interpretacja zapisu argumentu przez instrukcję if()

Warunek zostaje uznany za spełniony, gdy jego wartość wynosi **true**. Warunek zostaje uznany za niespełniony, gdy jego wartość wynosi **false**. W praktyce jednak nie musimy operować wyłącznie na wartościach **true/false** czy **1/0**. W rzeczywistości za fałsz (**false**) jest uznawana każda wartość reprezentująca zero. Za prawdę (**true**) jest uznana każda wartość, która jest różna od zera.

W poniższym kodzie nie ma bezpośredniego zapisu warunku porównującego zmienną „a” do zera. Chociaż zmienna „a” jest liczbą całkowitą, to jej interpretacja może być dokonana w kategorii **true/false**.

```

int a;
cin >> a;

if (a)
{
    // Kod programu dla PRAWDY, gdy „a” będzie różne od zera }
}
{
    // Kod programu dla FAŁSZ, gdy „a” będzie równe zero }
}

```

7. Grupowanie warunków – budowa warunków złożonych

Każdy kolejny warunek podległy innemu warunkowi musimy wytabować (wciąć) stosownie do bloku kodu w jakim się ma znaleźć.

Jeśli kilka warunków występuje po sobie, możemy próbować łączyć je w jeden większy. Wymagane jest jednak, aby takie złożone warunki zapisywać z użyciem operatorów łączących:

&& - and – I – koniunkcja - oznacza, że argumenty po obu stronach muszą być prawdziwe, aby całość była prawdziwa.

|| - or – LUB – alternatywa - oznacza, że wystarczy żeby choć jeden z argumentów po którejś ze stron był prawdziwy, aby całość była prawdziwa.

Przykład sprawdzenia, czy liczba jest z przedziału od 2 do 4 włącznie: $2 \leq a$

```

{
    if (a <= 4)
    {
        cout << "W zakresie";
    }
    else

```

```

{
    cout << "Pozna zamkresom";
}
else
{
    cout << "Pozna zamkresom";
}

```

Możemy zredukować powyższy kod do zapisu:

```

if((2 <= a) && (a <= 4))
{
    cout << "W zamkresie";
}
else
{
    cout << "Pozna zamkresom";
}

```

Operatory **&&** oraz **||** są to **operatory logiczne** stosowane w warunkach. Operują one na wartościach **true/false**.

Operatory **&** oraz **|** są to **operatory bitowe** stosowane w obliczeniach. Operują one na poszczególnych bitach zmiennych.

Pomylenie operatora bitowego z logicznym, może dać całkiem inny wynik operacji, a tym samym warunek wykona się nie tak jak planowaliśmy.

<pre> if(4 & 2) { cout << "NIGDY sie nie wykonalo!"; } else { cout << "ZAWSZE sie wykonalo!"; } </pre>	<pre> if(4 && 2) { cout << "ZAWSZE sie wykonalo!"; } else { cout << "NIGDY sie nie wykonalo!"; } </pre>
--	---

Operator dodatkowy:

^ - xor - ALBO – alternatywa wykluczająca- oznacza, że tylko jeden argument po którejś stronie musi być prawdziwy (prawdziwość obydwu argumentów jednocześnie jest traktowana jako całościowa nieprawda).

Uwaga! Operator xor jest **operatorem bitowym**. Aby go umiejętnie wykorzystać w warunku (jako operator logiczny), należy go używać wyłącznie na zmiennych logicznych lub zmienne innego typu „zredukować” do interpretacji **true/false**.

Uwaga! W języku c operator **^** **nie jest** operatorem podnoszenia do potęgi (choć przyzwyczailiśmy się, że ten symbol oznacza taką właśnie operację).

```

bool bLewo = true;
bool bPrawo = false;

cout << "Gora lodowa przed nami!" << endl;

```

```

if( ( (bZewo ~ bPrawo) ) == true )
{
    cout << "Uff, udało się ominąć" << endl;
}
else
{
    cout << "Direkt zatonął!" << endl;
}

```

W przypadku konstruowania warunku bardzo złożonego, z dużej liczby warunków pomniejszych (można więcej niż tylko dwóch), zalecane jest używanie nawiasów przy ich zapisie, aby łatwiej rozpoznać o co chodzi.

Nie należy zapisywać warunków złożonych jako np.:

```
if( a < b < c )
```

Zamiast tego należy taki warunek rozdzielić na 2 mniejsze zapisane: `if( ( a < b ) && ( b < c ) )`

Ten pierwszy warunek ma sens wyłącznie matematyczny.

Kompilator nie jest w stanie zrozumieć pierwszego zapisu.

Procesor jest w stanie zrealizować naraz jedną operację na maksymalnie dwóch argumentach (mówiąc w wielkim uproszczeniu).

8. Warunek krótki - (?:)

Dla prostych operacji możemy użyć „warunku krótkiego”. Nie wykona on skomplikowanego kodu wewnątrz siebie ale jest przydatny np. do podstawiania wartości zmiennym. (nawiasy są jedynie elementem stylistycznym, nie trzeba ich stosować).

(warunek ? jeśli prawda, zrób to : jeśli fałsz, wykonaj to)

Poniższy kod:

```

if( a == 0 )
{
    cout << "ZERO!";
}
else
{
    cout << "Rozne od zera";
}

```

... można zapisać skrótowo jako:

```
cout << ( a == 0 ? "ZERO!" : "Rozne od zera" );
```